Making Shallow Learning Work: Achieving Convergence and Generalization

Danna Gurari
University of Colorado Boulder
Spring 2026

BY NC SA

https://dannagurari.colorado.edu/course/neural-networks-and-deep-learning-spring-2026/

Review

- Last lecture:
  - Issue: input shifts cause fully connected networks to fail
  - Idea: convolutional neural networks (CNNs) designed for translation invariance
  - Implementation: training CNNs end-to-end with gradient descent
  - Real-world success: LeNet-5 for check reading
  - Programming tutorial: convolutional neural networks in PyTorch
- Assignments (Canvas):
  - Problem set 1 due Friday
- Questions?

Today's Topics

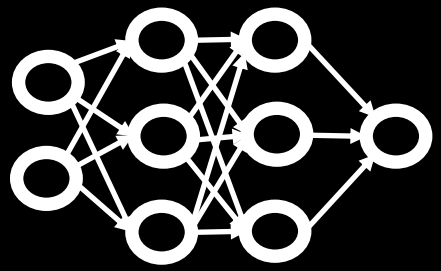
- Shallow learning: why doesn't it work?
- Navigating the loss landscape: how to make learning converge
- Generalizing to the real world: avoiding overfitting and underfitting
- Programming tutorial

Today's Topics

- Shallow learning: why doesn't it work?
- Navigating the loss landscape: how to make learning converge
- Generalizing to the real world: avoiding overfitting and underfitting
- Programming tutorial

Context: Shallow Learning

- Architecture: typically 1 hidden layer with ~5-100 neurons and occasionally 2 layers; e.g.,



Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

- Dataset: typically constrained to toy datasets (e.g., XOR) or relatively small, task-specific datasets (e.g., MNIST)
- Hardware: memory-constrained (e.g., KBs vs today's GBs); slow data loading (e.g., from floppy disks)
- Software: no tailored programming language (e.g., C or Fortran); often no parallel computing
- Training duration: regularly took days



e.g., floppy disk



e.g., SUN-3 workstation

Gap Between Theory and Practice

- In theory, neural networks can learn any function (with a single hidden layer)
- In practice, training often fails to converge or produce useful solutions

(Technical Report, 1998)

Efficient BackProp

Yann LeCun[1], Leon Bottou[1], Genevieve B. Orr[2], and Klaus-Robert Müller[3]

"Backpropagation… can seem more an art than a science. Designing and training a network using backprop requires making many seemingly arbitrary choices such as the number and type of nodes, layers, learning rates, training and test sets, and so forth… there are heuristics… that can help guide a practitioner to make better choices."

What Are Reasonable Strategies? - Empirical Regularities

Hyperparameters (selected); e.g.,

- Number of layers
- Number of units in each layer
- Activation function
- Learning rate
- Batch size
- …

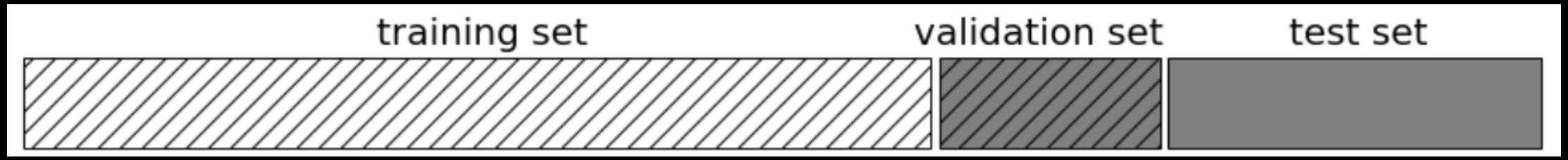
Parameters (learned)

- Weights
- Biases
- …

Challenge: how to choose hyperparameters without repeatedly observing the test data (which is cheating)?

Hyperparameters Evaluated with Validation Set

- Dataset divided into three splits; e.g., using a 60%/20%/20% train/val/test split

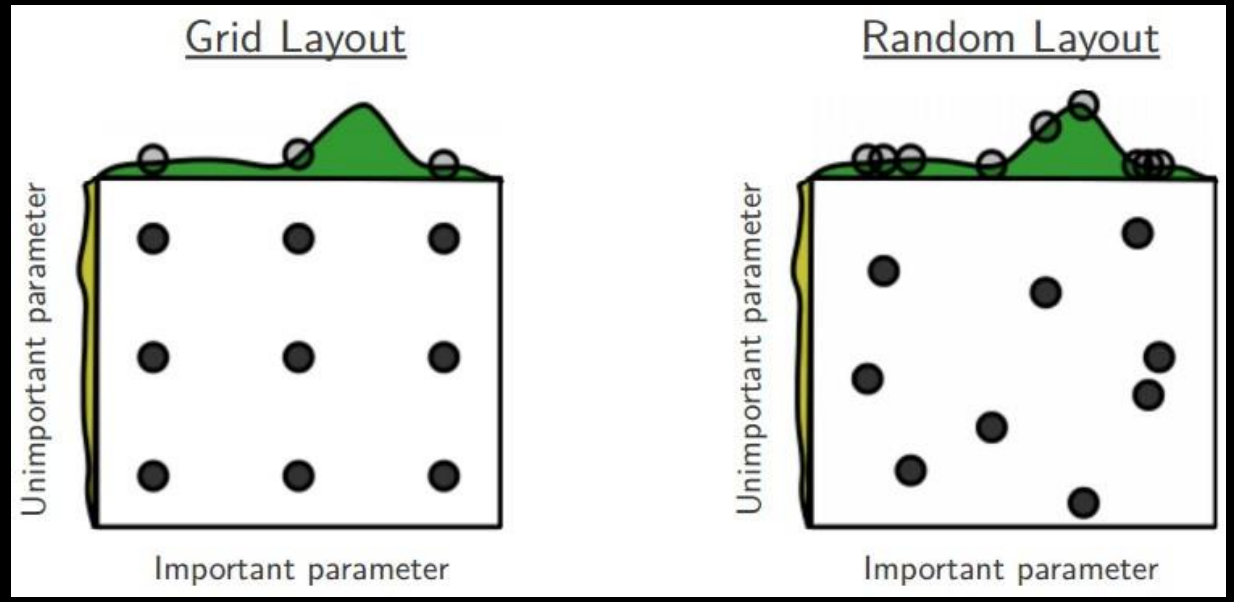


- **Hyperparameter analysis**: evaluate using validation set
- **Final model**: train on training AND validation splits

https://github.com/amueller/introduction_to_ml_with_python/blob/master/05-model-evaluation-and-improvement.ipynb

Hyperparameter Tuning Approaches

- Automatic



As exemplified, grid search is inferior when selecting multiple hyperparameters since **fewer values are tested** for each (in this example, two hyperparameters); random search better supports identifying impactful hyperparameters

- Manual

Bergstra et al; Random Search for Hyper-Parameter Optimization, 2012

Today's Topics

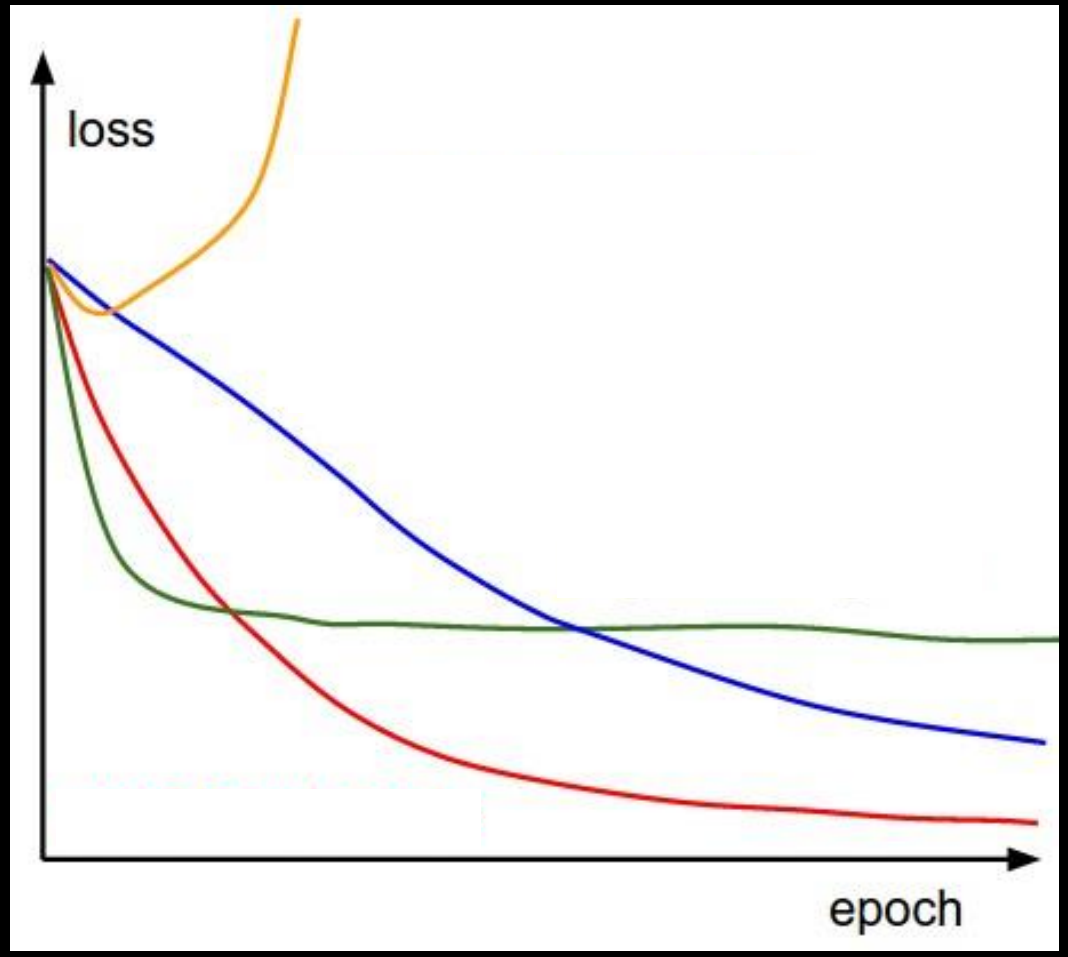
- Shallow learning: why doesn't it work?
- Navigating the loss landscape: how to make learning converge
- Generalizing to the real world: avoiding overfitting and underfitting
- Programming tutorial

Training Entails Babysitting Loss Curves…

Practitioners ask themselves “How is learning going based on the training loss curve?”

From the four curves, which represents learning progressing the worst? best?

- Curve 1
- Curve 2
- Curve 3
- Curve 4



https://cs231n.github.io/neural-networks-3/#update

What is a Good Loss Value?

- 0… no error ☺
- In practice, a value better than the *expected* one for the loss function
  - e.g., What would be expected for cross-entropy loss? When assuming equal probability per class for K classes, random guessing gives: Loss = - ln (1/K) = ln (K)

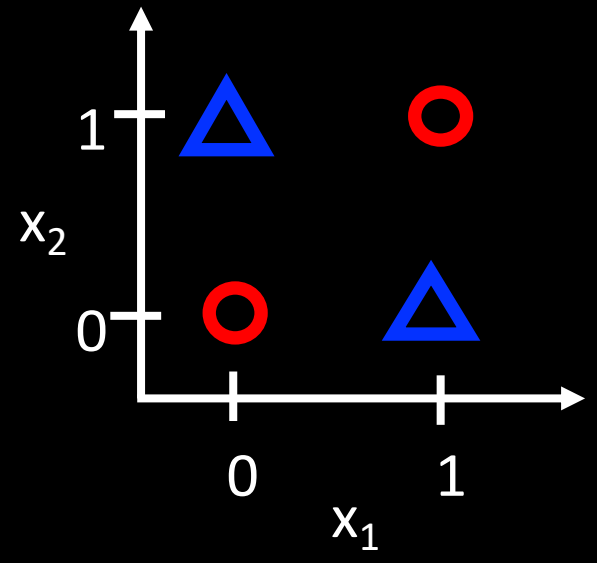
Key Issues for Effective Learning: Dataset, Architecture, & Training

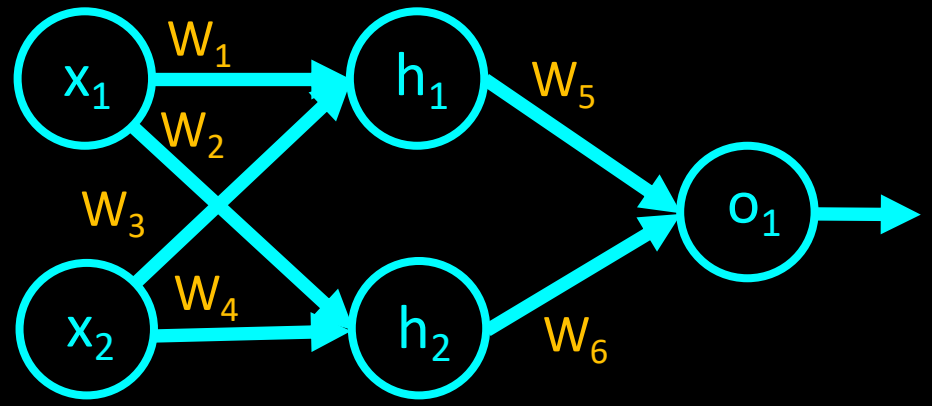
1. Input data

2. Model design

3. Initial weights

4. Gradient-based updates





Issue 1: Input Data's Impact on Learning

Intuition: imagine driving down a road with a car that has a steering wheel that only takes "hard" rights and lefts and is capped at 5 miles per hour; how would the journey look?



https://www.reddit.com/r/BMW/comments/zkkef1/before_and_after/

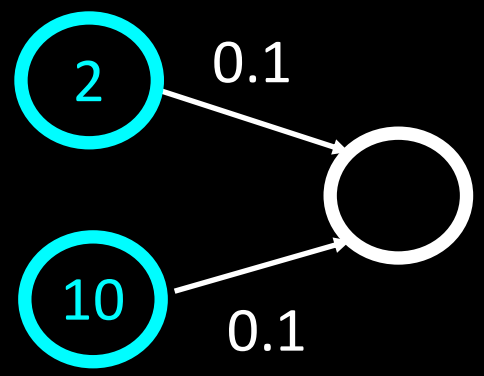


https://texomaexcavationandconstruction.com/gravel-roads-construction-forest-wooded-remote-areas/

Issue 1: Input Data's Impact on Learning

Extreme case illustrating issue: all inputs of the same sign
- slow learning can stem from inefficient, zig-zagging from shared gradient sign;

e.g.,



(Targe value = 0)

Additional inputs:
[5, 24]
[10, 31]

Forward pass: sigmoid(0.1 x 2 + 0.1 x 10) = 0.77

Backward pass:
(recall derivation from lecture 3)

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial o} x \frac{\partial o}{\partial z} x \frac{\partial z}{\partial w_i}$$

$$\frac{\partial E}{\partial w_i} = -(t - o_6)\ x\ (1- o_6)o_6\ x\ x_i$$

Input sign drives gradient direction

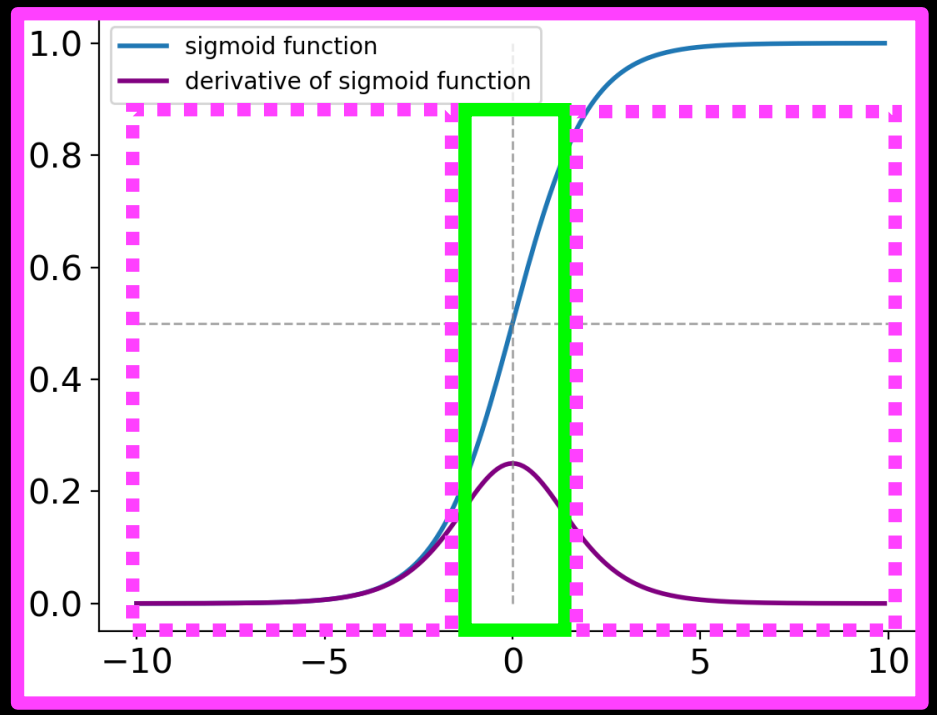
$$= -(0 - 0.77)\ x\ (1- 0.77)0.77\ x\ x_i = 0.14\ x\ x_i$$

$$\frac{\partial E}{\partial w_1} = 0.28 \qquad \frac{\partial E}{\partial w_2} = 1.4$$

Parameter updates: What direction will the weight updates move?
Parameter updates: If all inputs are instead negative, what will be the direction?
Will these update trends change with additional inputs that share the same sign?
- No; updates must zig-zag between two directions to arrive at the optimal direction

Issue 1: Input Data's Impact on Learning

Extreme case illustrating issue: all inputs of the same sign
- slow learning can stem from inefficient, zig-zagging from shared gradient sign and small gradients



https://medium.com/@irenemarkelic/the-sigmoid-and-its-derivative-40b44485ca48

What are ideal weighted sums for the activation function? Why?

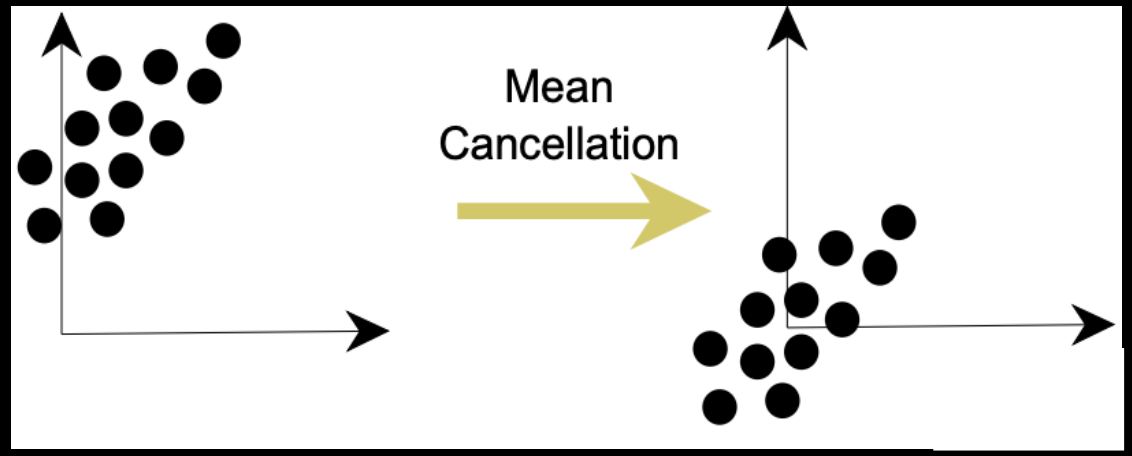
What are expected weight sums when all inputs share the same sign?

Idea: Preprocess Input to Accelerate Learning

Extreme case illustrating issue: all inputs of the same sign
- slow learning can stem from inefficient, zig-zagging from shared gradient sign and small gradients

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)
e.g.,



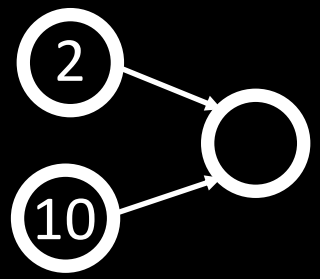
(LeCun et al, Technical Report, 1998)

Idea: Preprocess Input to Accelerate Learning

Extreme case illustrating issue: all inputs of the same sign
- slow learning can stem from inefficient, zig-zagging from shared gradient sign and small gradients

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)
e.g.,



Additional inputs:
[5, 24]
[10, 31]

How many features are in the input?
- 2

What is the mean of feature 1?
- (2+5+10)/3 = 5.7

What are zero-centered values for feature 1?
- [2-5.7, 5-5.7, 10-5.7] = [-3.7, -0.7, 4.3]

What are zero-centered values for feature 2?
- [10-21.7, 24-21.7, 31-21.7] = [-11.7, 2.3, 9.3]

Idea: Preprocess Input to Accelerate Learning

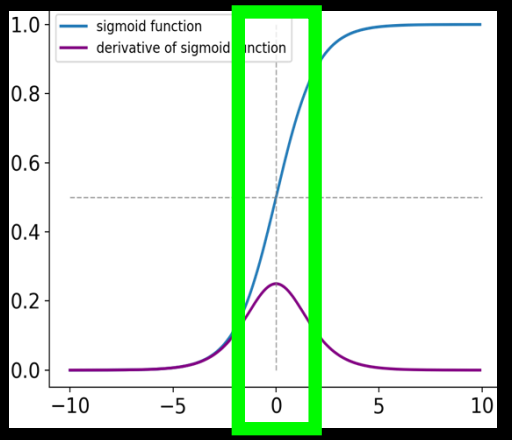
Extreme case illustrating issue: all inputs of the same sign
- slow learning can stem from inefficient, zig-zagging from shared gradient sign and small gradients

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)
e.g.,

Which inputs moved closer to the ideal 0 region?

- 1: [2, 10] vs [-3.7, -11.7]
  weighted sum: 1.2 vs -1.54
- 2: [5, 4] vs [-0.7, 2.3]
  weighted sum: 0.9 vs 0.16
- 3: [10, 31] vs [4.3, 9.3]
  weighted sum: 4.1 vs 1.36



https://medium.com/@irenemarkelic/the-sigmoid-and-its-derivative-40b44485ca48

Idea: Preprocess Input to Accelerate Learning

Extreme case illustrating issue: all inputs of the same sign
- slow learning can stem from inefficient, zig-zagging from shared gradient sign and small gradients

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)
e.g.,

Given the three centered inputs, will all parameter updates move in the same direction?

- 1: [-3.7 -11.7]
- 2: [-0.7, 2.3])
- 3: [4.3, 9.3]

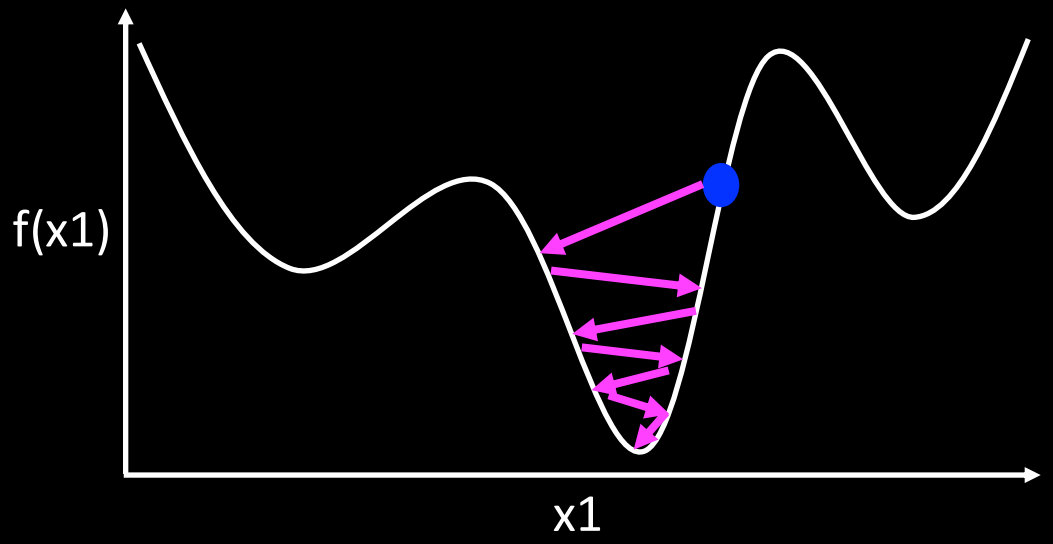
No! e.g., input 1 will increase all weights, input 3 will decrease all weights, and input 2 will move in both directions

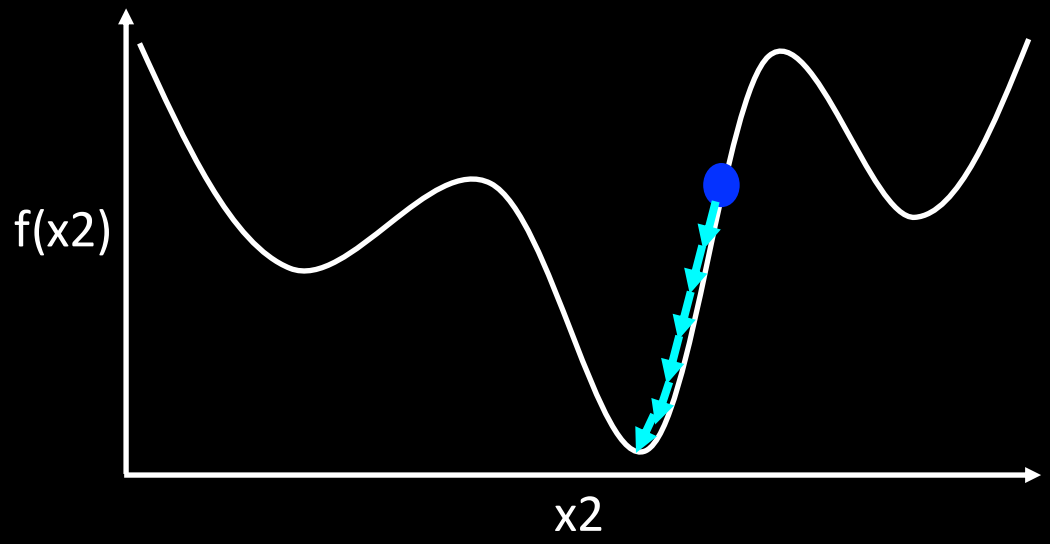
Idea: Preprocess Input to Accelerate Learning

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)

Another Problem: different feature scales cause differently scaled gradients, which translates to inefficient learning where parameter updates can overshoot in some cases and crawl in others (when using the same learning rate)





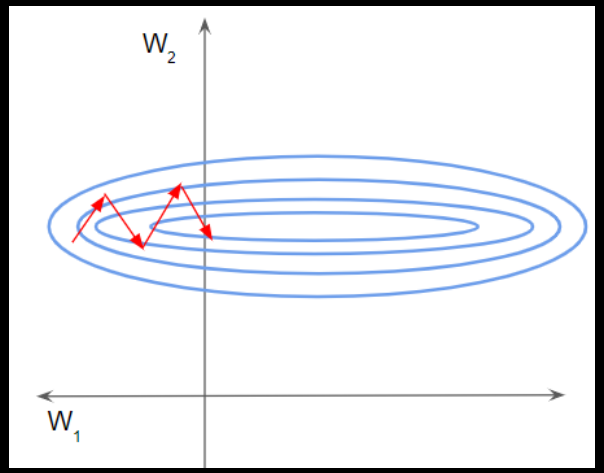
Idea: Preprocess Input to Accelerate Learning

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)

Another Problem: different feature scales cause differently scaled gradients, which translates to inefficient learning where parameter updates can overshoot in some cases and crawl in others (when using the same learning rate)

Loss landscape:



https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739

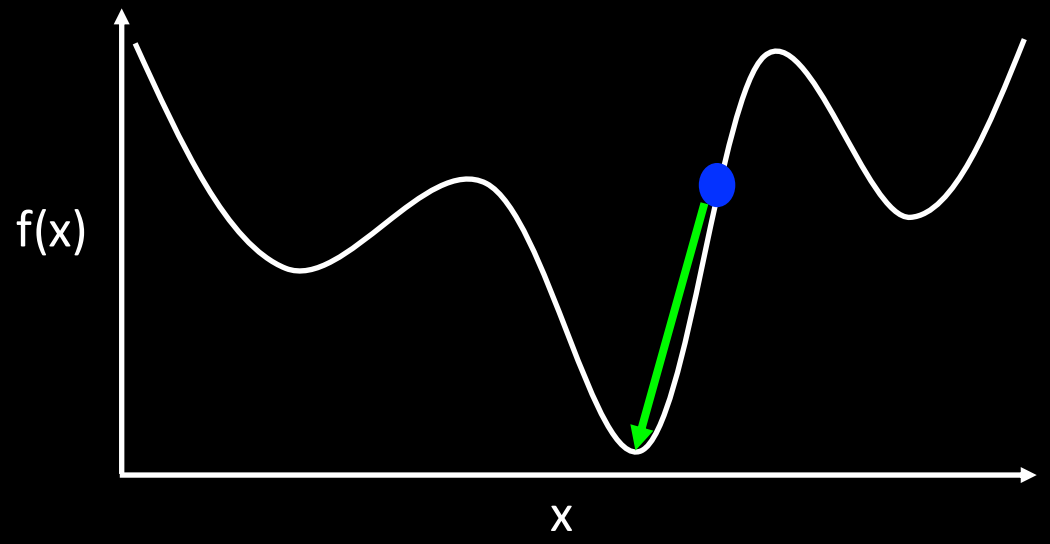
Idea: Preprocess Input to Accelerate Learning

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)

Another Problem: different feature scales cause differently scaled gradients, which translates to inefficient learning where parameter updates can overshoot in some cases and crawl in others (when using the same learning rate)

Next Solution: scale features to the same scale
i.e., then all features can update more efficiently
(when choosing a good learning rate)



Idea: Preprocess Input to Accelerate Learning

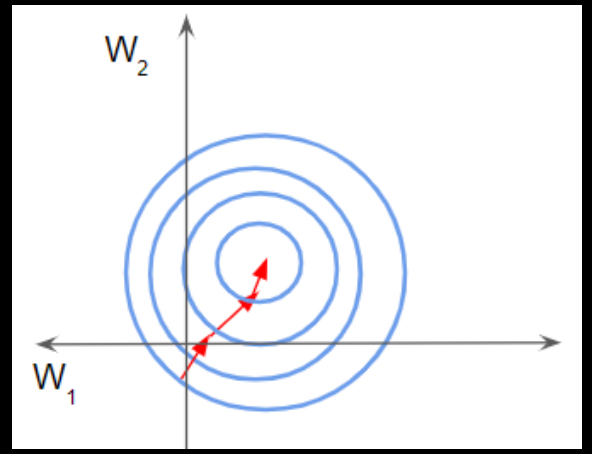
General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)

Another Problem: different feature scales cause differently scaled gradients, which translates to inefficient learning where parameter updates can overshoot in some cases and crawl in others (when using the same learning rate)

Next Solution: scale features to the same scale
i.e., then all features can update more efficiently
(when choosing a good learning rate)

Loss landscape:



https://towardsdatascience.com/batch-norm-explained-visually-how-it-works-and-why-neural-networks-need-it-b18919692739

Idea: Preprocess Input to Accelerate Learning

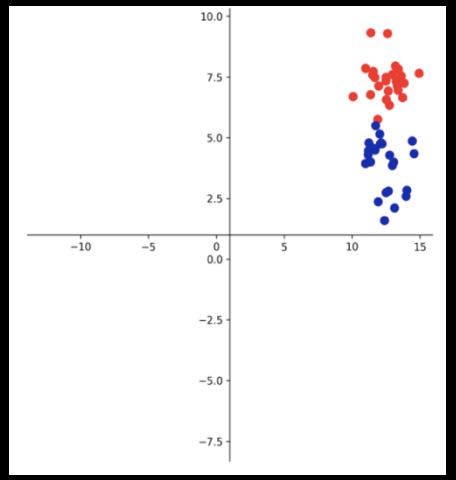
General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)

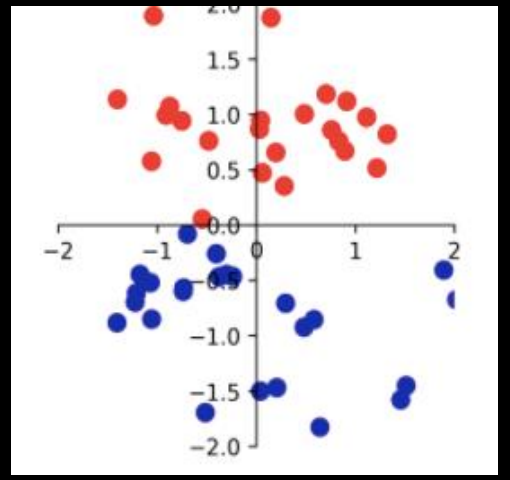
Another Problem: different feature scales cause differently scaled gradients, which translates to inefficient learning where parameter updates can overshoot in some cases and crawl in others (when using the same learning rate)

Next Solution: scale features to the same scale by dividing each by its standard deviation in the training data

Original:



Standardized:



Note: test features also should be scaled using the values learned on the training data

Note: standardization is often called "normalization" for training neural networks

https://github.com/amueller/introduction_to_ml_with_python/blob/main/03-unsupervised-learning.ipynb

Idea: Preprocess Input to Accelerate Learning

General case: non-zero-centered inputs slow learning by biasing weighted sums and parameter updates' direction

Solution: zero center the input; i.e., calculate mean for every feature in the training set and then subtract resulting values from every input feature (causes every feature to have no directional bias across training set)

Another Problem: different feature scales cause differently scaled gradients, which translates to inefficient learning where parameter updates can overshoot in some cases and crawl in others (when using the same learning rate)

Next Solution: scale features to the same scale by dividing each by its standard deviation in the training data
e.g.,

Centered inputs
- 1: [-3.7 -11.7]
- 2: [-0.7, 2.3])
- 3: [4.3, 9.3]

How many features are in the input?
- 2

What is the standard deviation of input feature 1?

$$\sigma = \sqrt{((-3.7)^2+(-0.7)^2 + (4.3)^2)/3} = 3.3$$

What is the standard deviation of input feature 2?

$$\sigma = \sqrt{((-11.7)^2+(2.3)^2 + (9.3)^2)/3} = 8.7$$

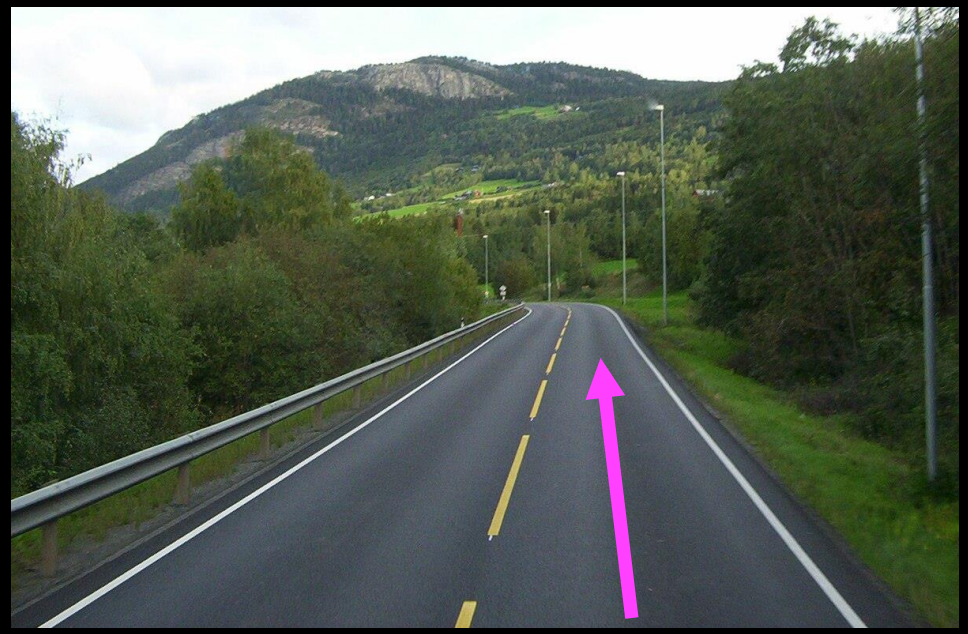
What are the inputs' standardized values?
- 1: [-3.7/3.3, -11.7/8.7] -> [1.1, 1.3]
- 2: [-0.7/3.3, 2.3/8.7] -> [-0.2, 0.26]
- 3: [4.3/3.3, 9.3/8.7] -> [1.3, 1.1]

Summary: Preprocessing Input Accelerates Learning

Intuition: enables driving down a road with a car that can go fast with a steering wheel that can go in the optimal direction



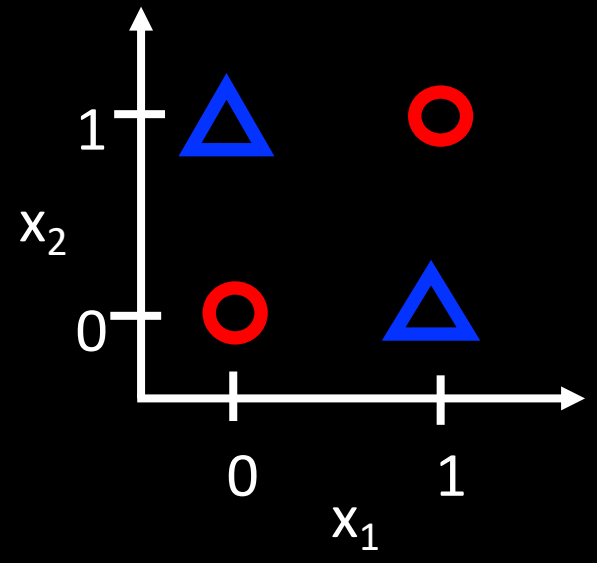
https://www.reddit.com/r/BMW/comments/zkkef1/before_and_after/



https://texomaexcavationandconstruction.com/gravel-roads-construction-forest-wooded-remote-areas/

Key Issues for Effective Learning: Dataset, Architecture, & Training

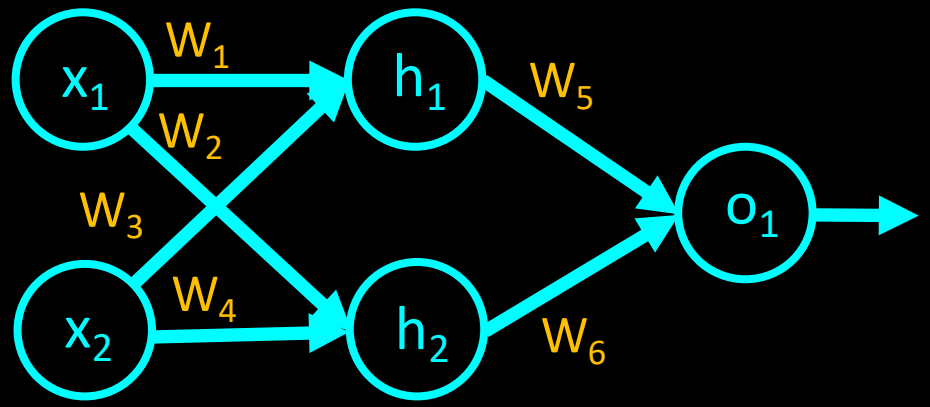
1. Input data



2. Model design

3. Initial weights

4. Gradient-based updates



Issue 2: Activation Function Can Slow Learning

Recall intuition: imagine driving down a road with a car that has a steering wheel that only takes “hard” rights and lefts and is capped at 5 miles per hour; how would the journey look?



https://www.reddit.com/r/BMW/comments/zkkef1/before_and_after/



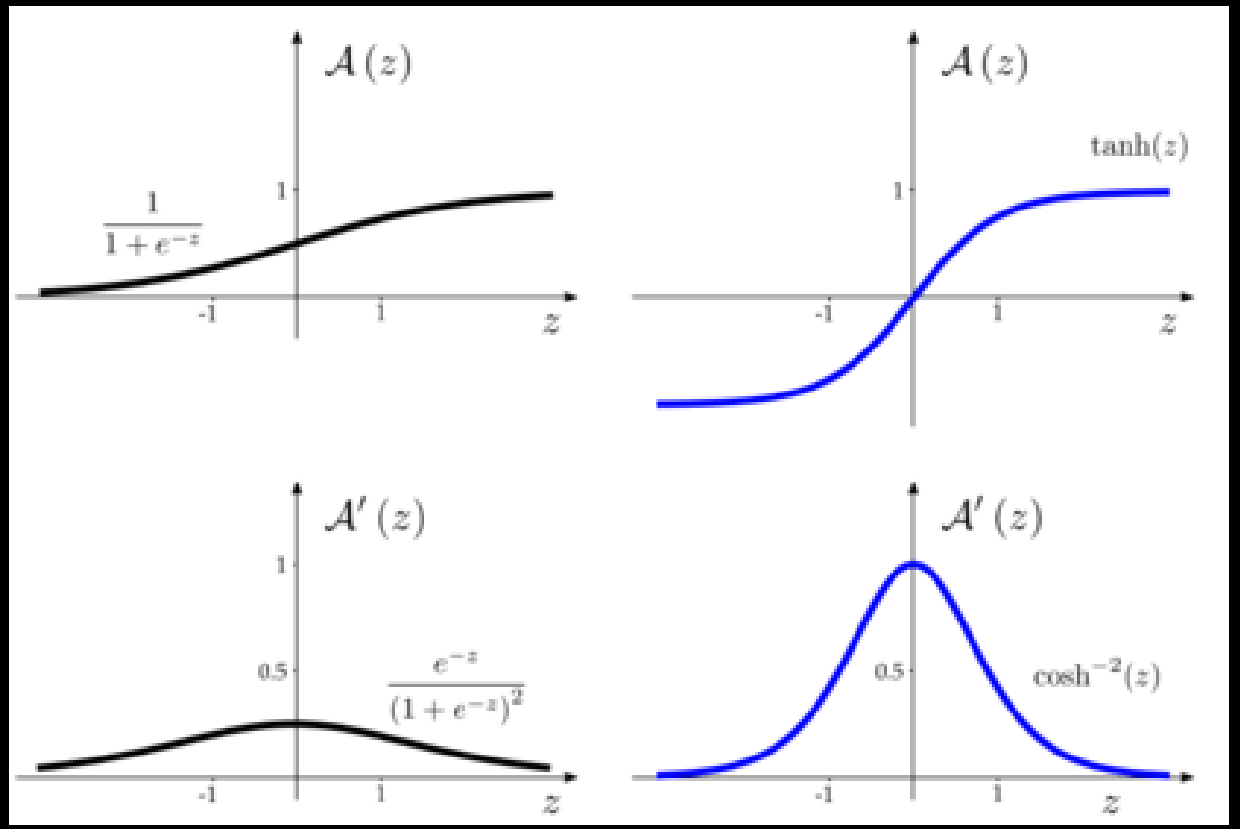
https://texomaexcavationandconstruction.com/gravel-roads-construction-forest-wooded-remote-areas/

Idea: Choose Activation Function that Accelerates Learning

- Sigmoid and tanh mimic neurons' "firing" behavior; how do they differ?
- What is the sign of:
  - sigmoid activations?
  - tanh activations?
  - Which is better for learning (recall, neuron inputs with all the same sign cause inefficient, zig-zagging learning)
- What is the range of gradient values for:
  - sigmoid?
  - tanh?
  - Which has values better for learning?

Sigmoid

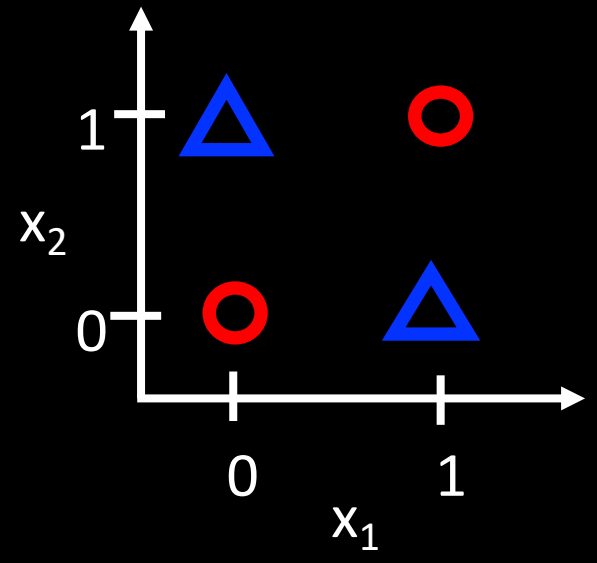
Tanh



Masi et al. Journal of the Mechanics and Physics of Solids. 2021

Key Issues for Effective Learning: Dataset, Architecture, & Training

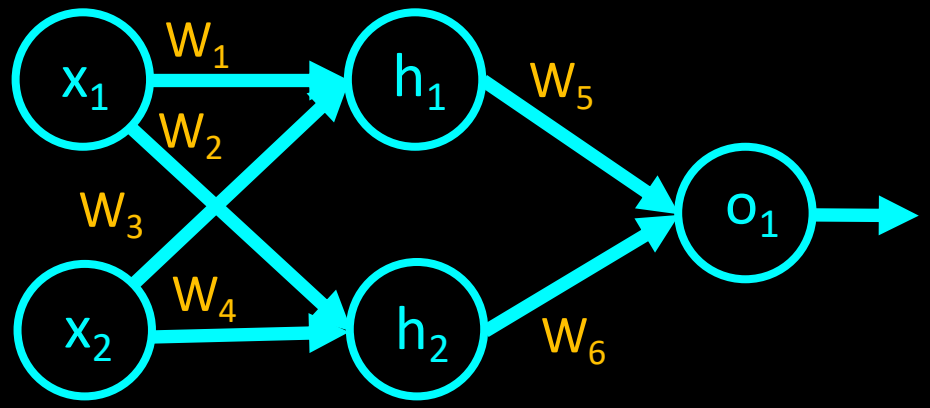
1. Input data



2. Model design

3. Initial weights

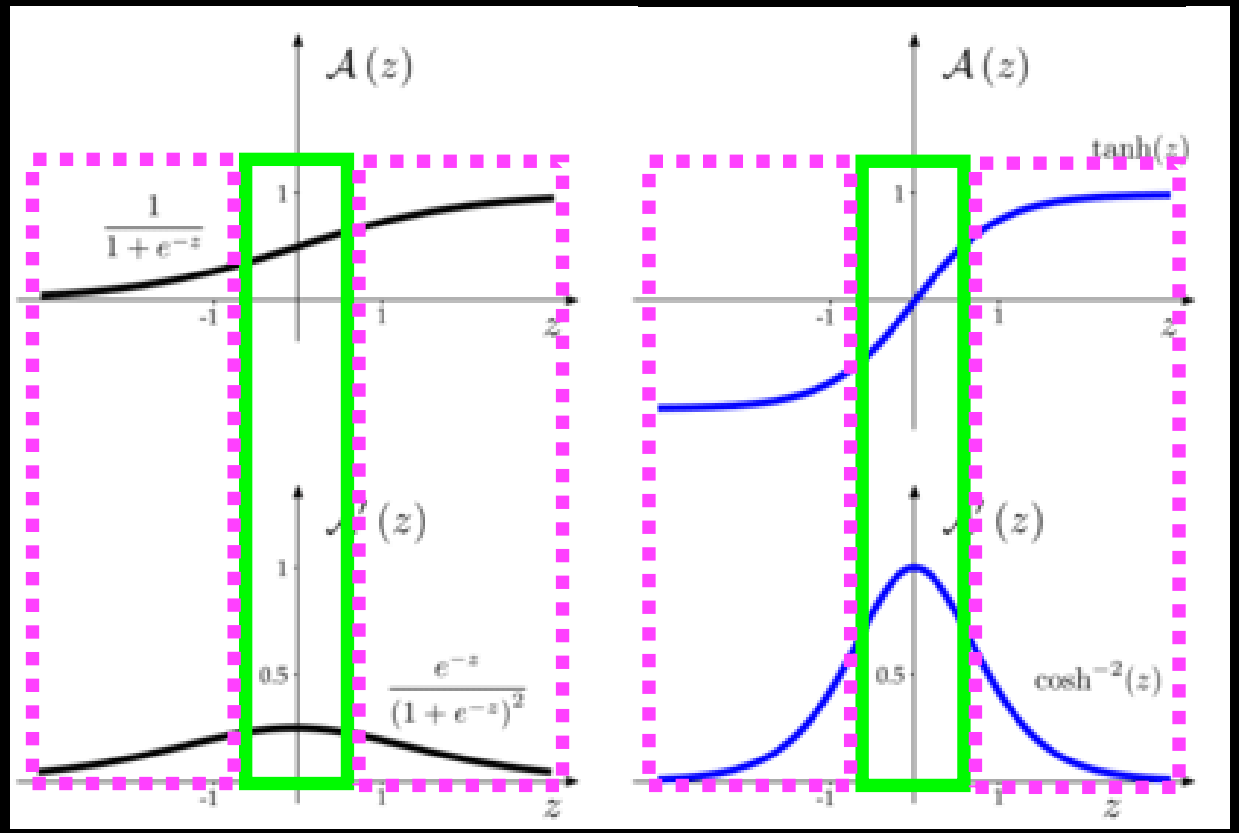
4. Gradient-based updates



Issue 3: Initial Parameter Values Can Slow or Block Learning

Sigmoid

Tanh



Neurons receiving very large or small "z" values have slow/no learning; why?

- small derivatives limit amount model parameters can change

Ideal: weighted sums cause derivatives that lie in the "good range"

Masi et al. Journal of the Mechanics and Physics of Solids. 2021

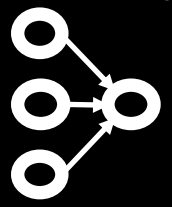
Idea: Initialize Parameters to Accelerate Learning

Approach: establish weighted sums with good gradients for learning

- Weights set to random, small, where the scale of "small" is key
- Biases set to 0

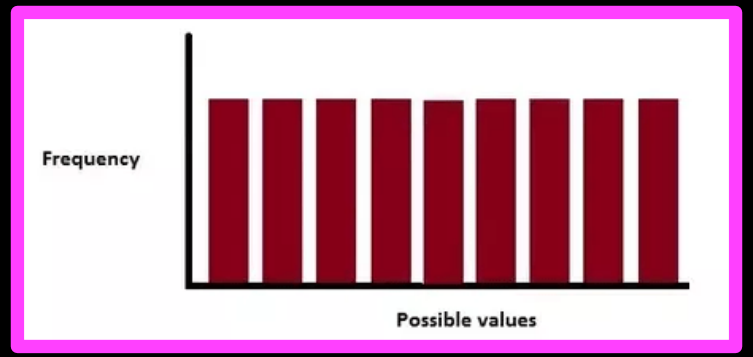
Weight initialization heuristics:

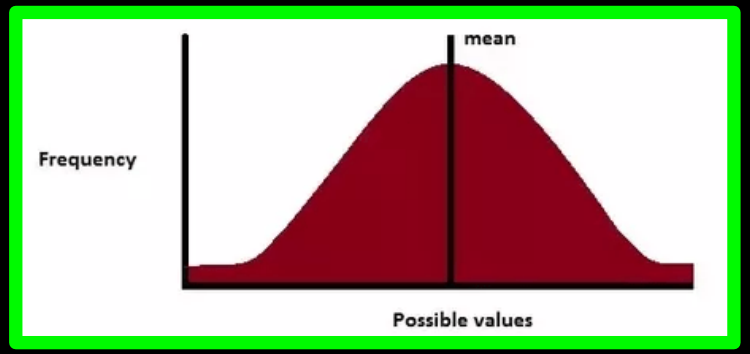
- 1986 backprop paper: randomly draw from uniform distribution between -0.3 and 0.3
- 1998 "Efficient backprop" paper: randomly draw from uniform distribution with mean 0 and standard deviation determine by the number of incoming connections to a neuron ("fan-in"): $\frac{1}{\sqrt{m}}$
  e.g., what is m for this?



Recall from lab assignment 1:

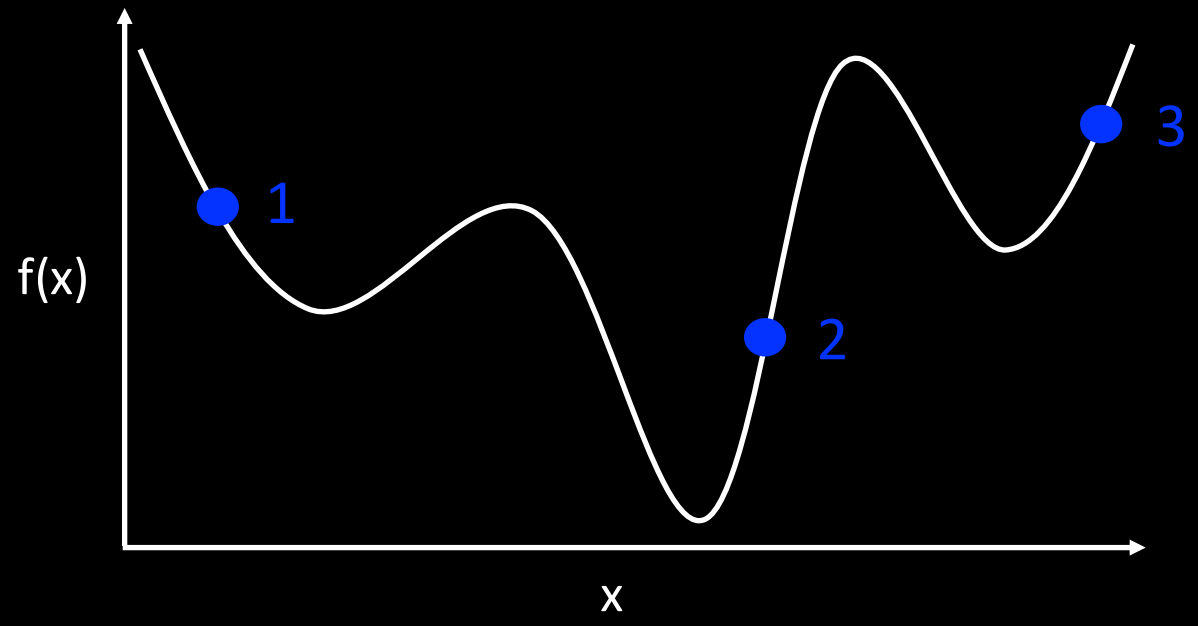
- Uniform vs Gaussian distribution?





https://deepdatascience.wordpress.com/2017/03/13/normal-vs-uniform-distribution/

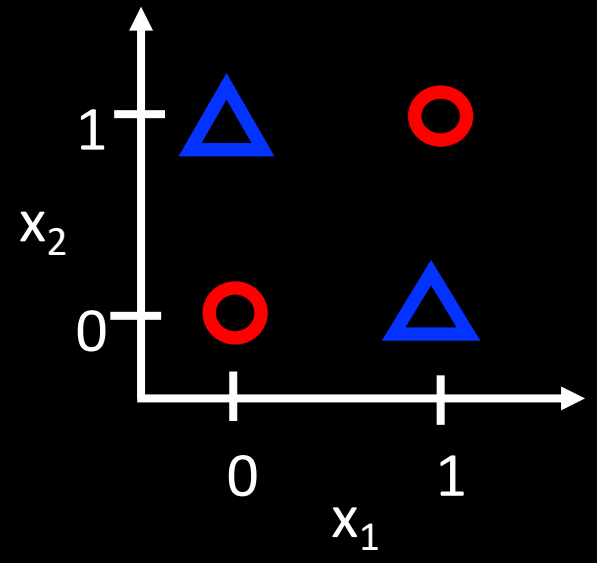
Idea: Initialize Parameters to Accelerate Learning



Practical note: we often repeat initialization
to try to arrive closer to a better solution

Key Issues for Effective Learning: Dataset, Architecture, & Training

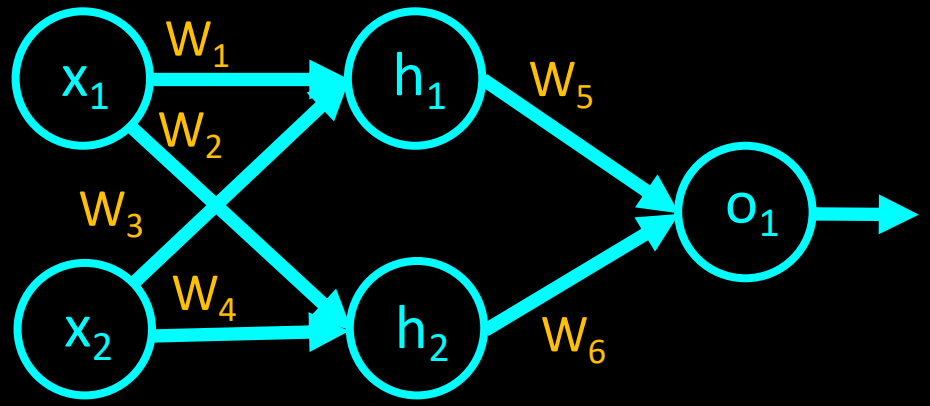
1. Input data



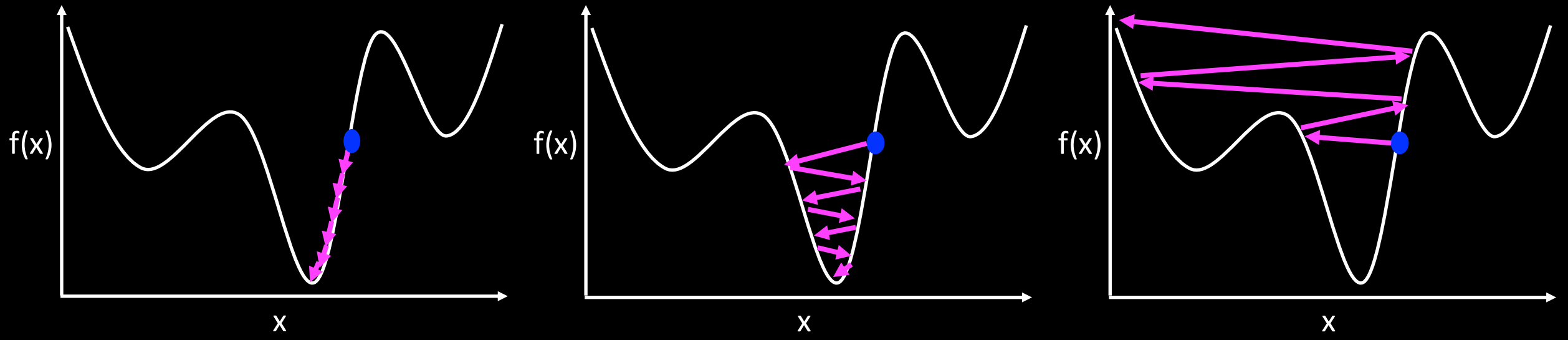
2. Model design

3. Initial weights

4. Gradient-based updates



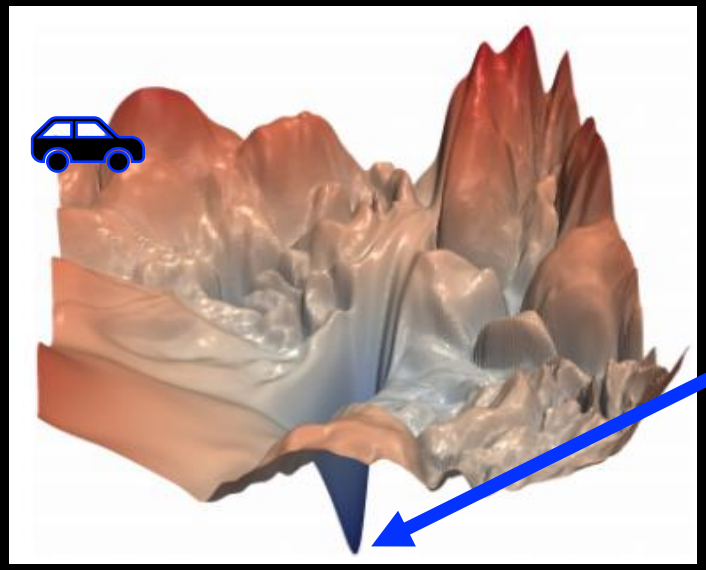
Issue 3: Learning is Inefficient or Fails



Issue 3: Learning is Inefficient or Fails

What can go wrong with gradient descent for models with 2 parameters?

(e.g., imagine descending error surface-vertical axis values-for all possible weight pair values in the horizontal plane)

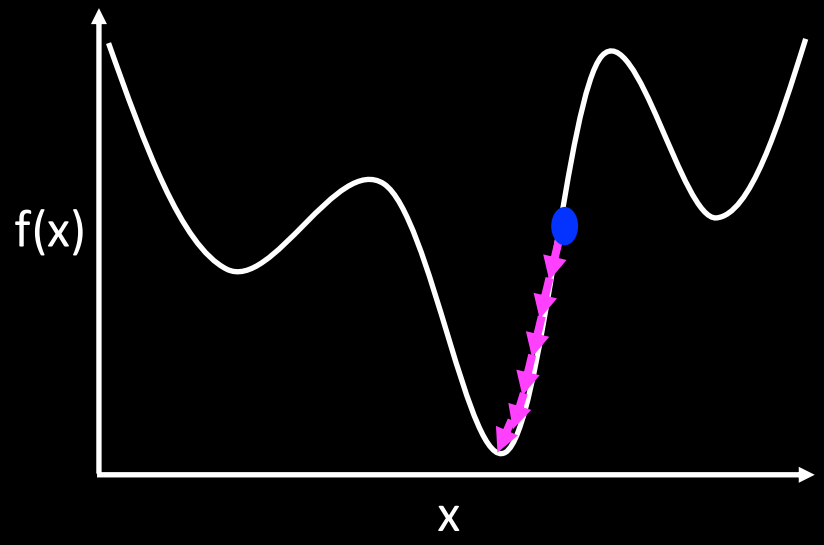


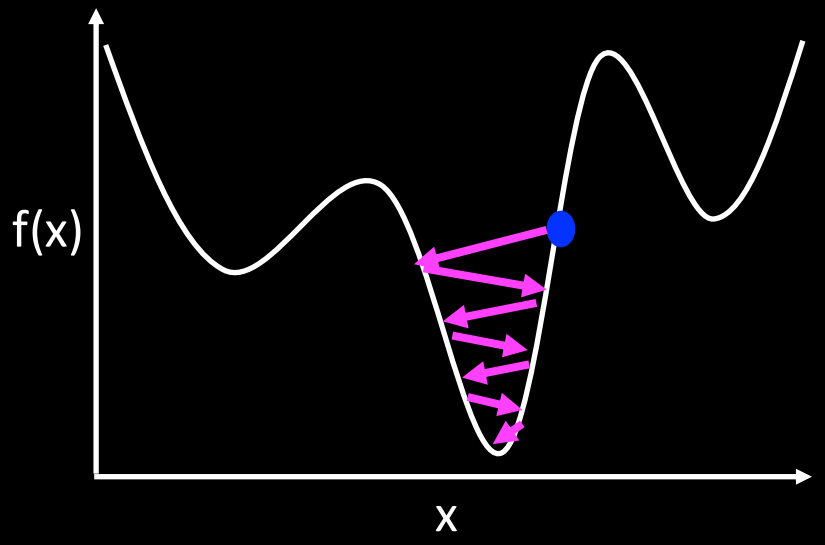
- get stuck in a ditch (local optimum)
- zig-zag on a ravine (little gradient)
- arrive at a flat plateau (no gradient)
- fail to arrive at a global minimum; this initially limited excitement about NNs

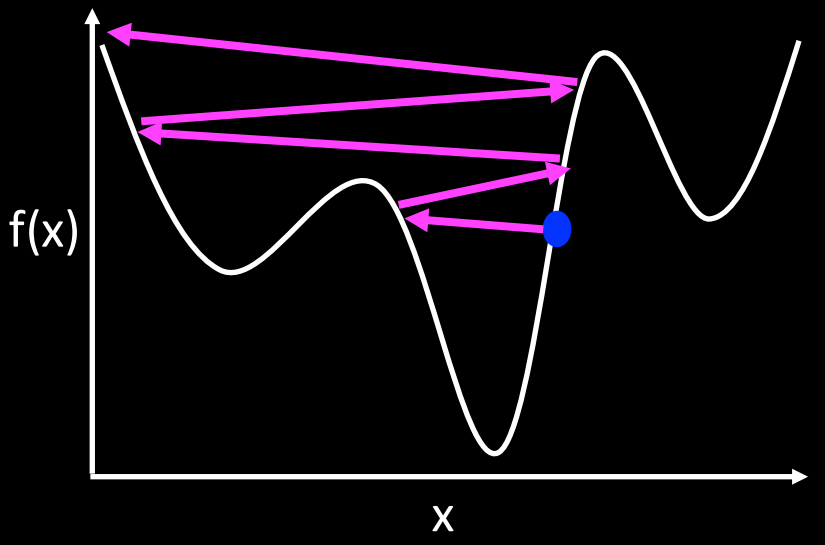
Li et al. Visualizing the Loss Landscape of Neural Nets. NeurIPS 2018.

Idea: Change Optimization Approach

How can we change the learning rate? $\mathrm{W} \leftarrow \mathrm{W} - \eta \frac{\partial L}{\partial \mathrm{W}}$







Increase if loss declines slowly

Decrease if loss oscillates, is stuck, or increases

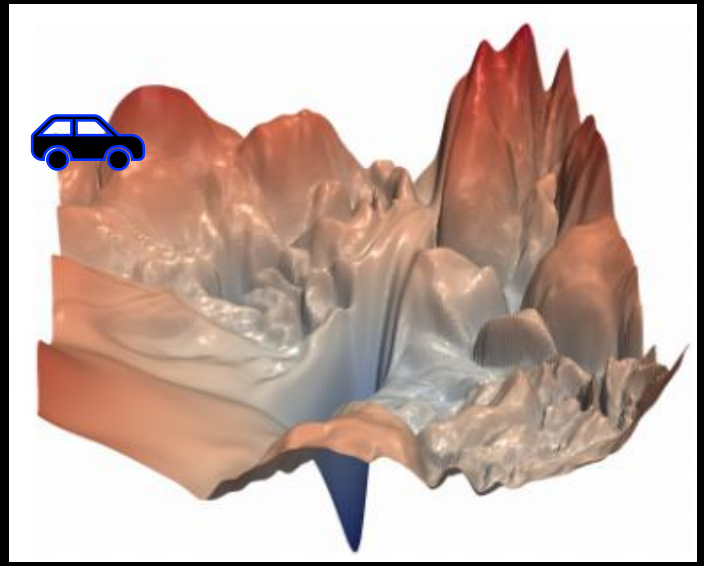
How many steps would it take to arrive at the minimum if we had the optimal learning rate?

Often, we decrease (anneal) the rate as more training epochs pass (e.g., linearly, exponentially)

Idea: Change Optimization Approach

What else might we change? $W \leftarrow W - \eta \frac{\partial L}{\partial W}$

Accumulate gradients across a batch of examples before updating parameters
- loss bounces less erratically than with stochastic learning (i.e., one example)



Li et al. Visualizing the Loss Landscape of Neural Nets. NeurIPS 2018.

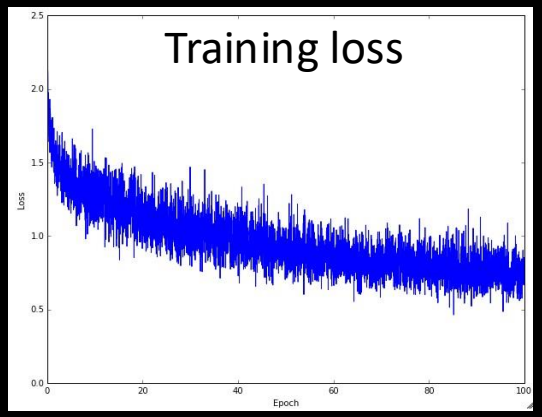
Assign a learning rate per parameter (e.g., larger for earlier layers, which receive smaller gradients)

Use gradient for acceleration (not speed), giving momentum to avoid local minima (e.g., imagine rolling a ball down a hill; the new velocity vector, v, captures cumulative direction of previous gradients)

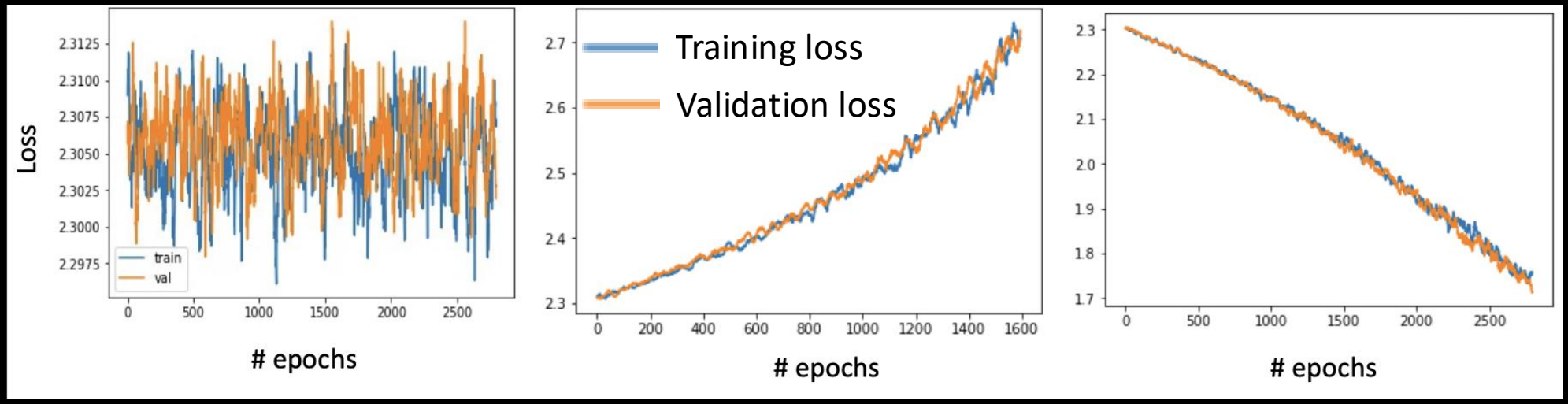
Ranges from 0 to 1 with larger values reflecting greater friction; need to be careful not to roll past and oscillate around minima

```
v = mu * v - learning_rate * dx # integrate velocity
x += v # integrate position
```

Discussion: From These Learning Curves, What Do You Think Is Happening and What Might Be a Fix?



(a)



(b) (c) (d)

https://cs231n.github.io/neural-networks-3/#update

Feeling Bewildered By Your Learning Curves?

You may feel better when looking at this link:

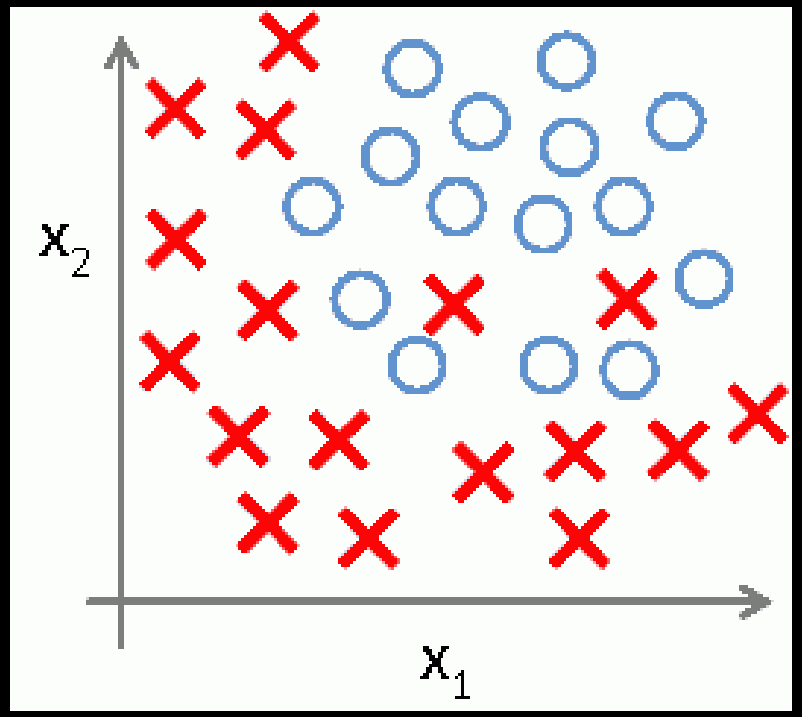
https://lossfunctions.tumblr.com/

Today's Topics

- Shallow learning: why doesn't it work?
- Navigating the loss landscape: how to make learning converge
- Generalizing to the real world: avoiding overfitting and underfitting
- Programming tutorial

Model Capacity: Recall Class Exercise

- Model-based approach: separate x from o



Class volunteer:
1) Draw a straight line (linear equation)
2) Draw a parabola (quadratic equation)
3) Draw any curve

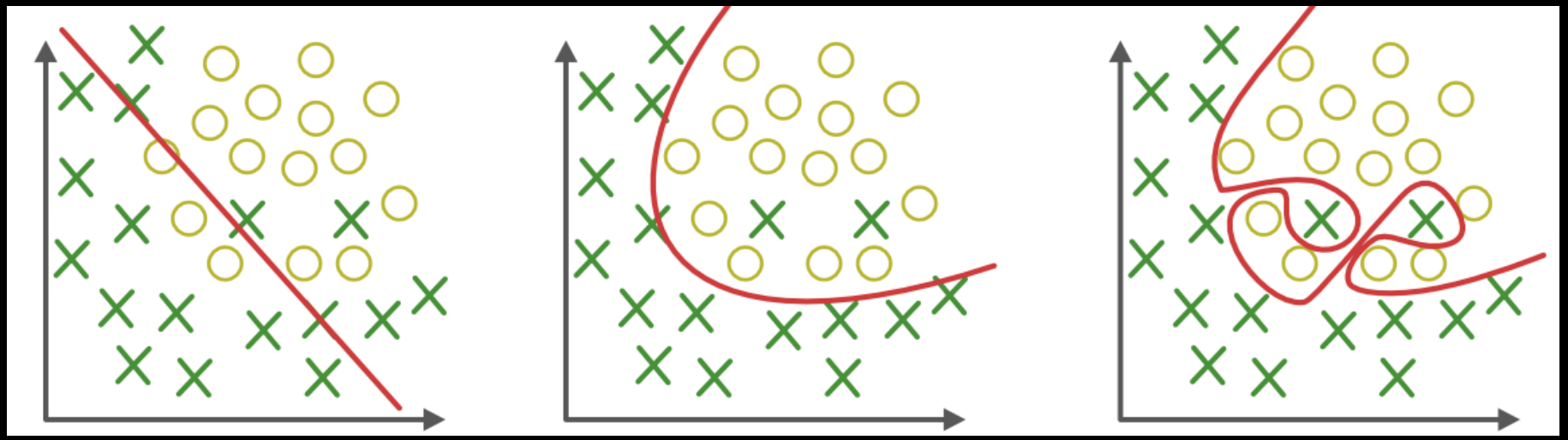
Models with increasing representational capacity

https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76

Model Capacity

Which model would you choose to separate x from o?

(a) (b) (c)

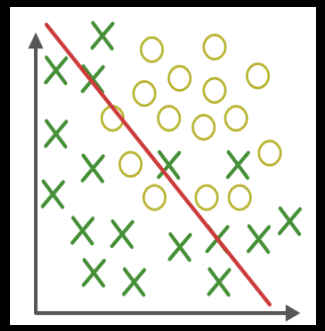


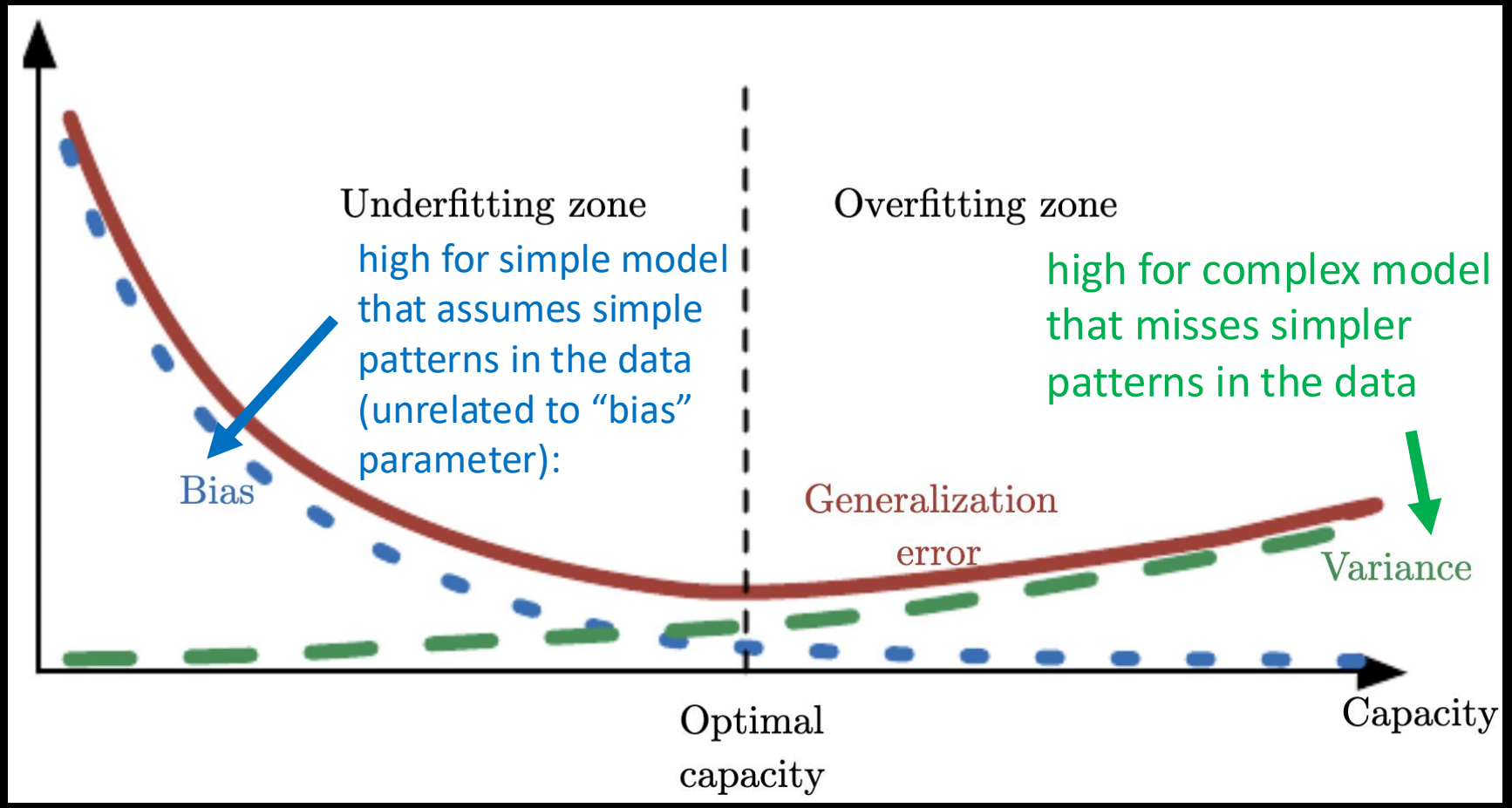
Underfits: too simple to explain the data

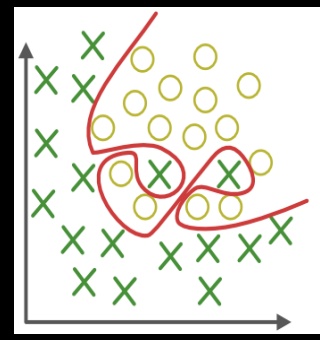
Overfits: too complex to generalize to a test set

https://towardsdatascience.com/underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6fe4a8a49dbf

Model Capacity: Overfitting vs Underfitting



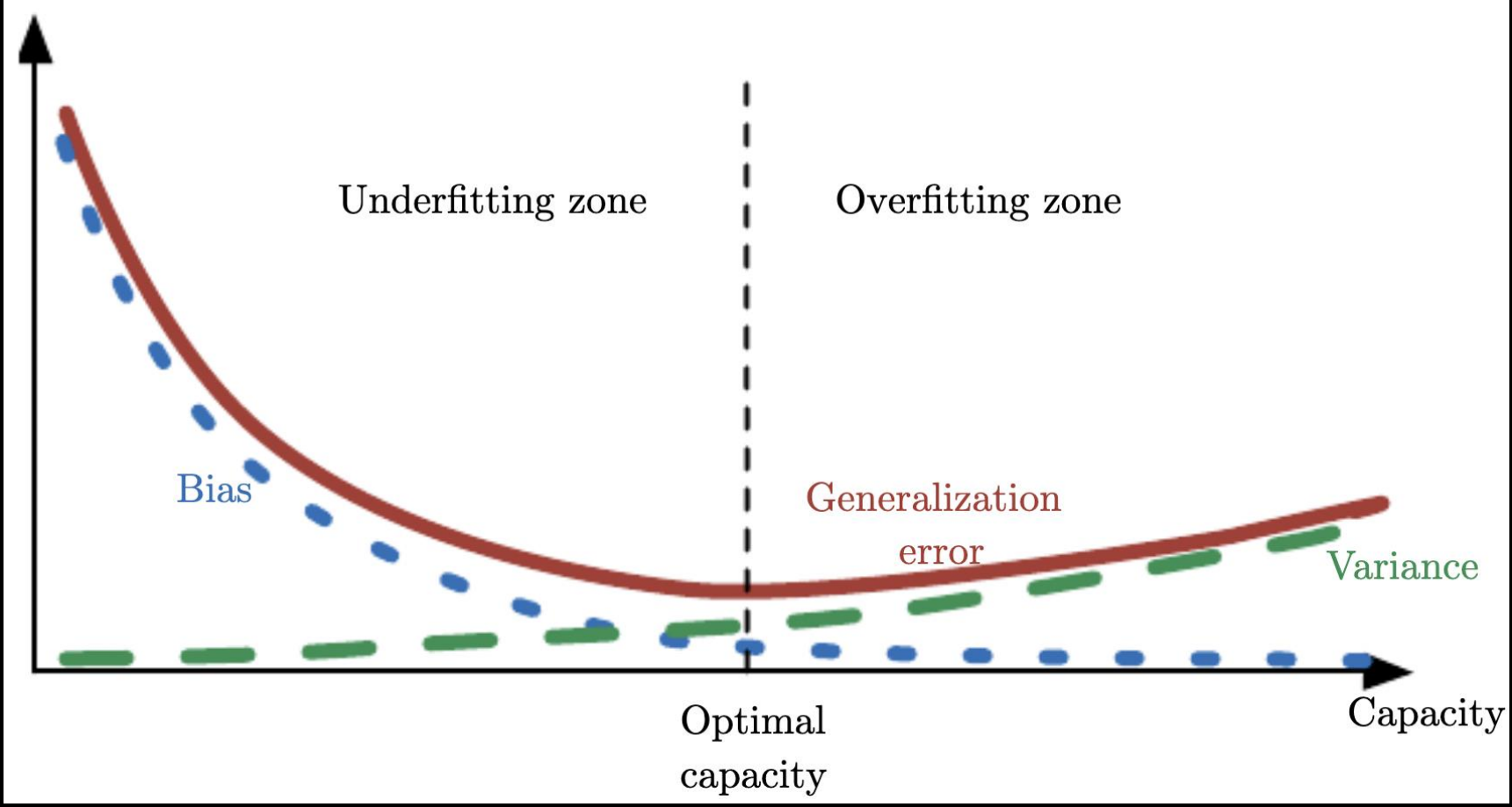




Often discussed with respect to a **bias-variance** trade-off

Ian Goodfellow, Yoshua Bengio, and Aaron Courville; Deep Learning, 2016

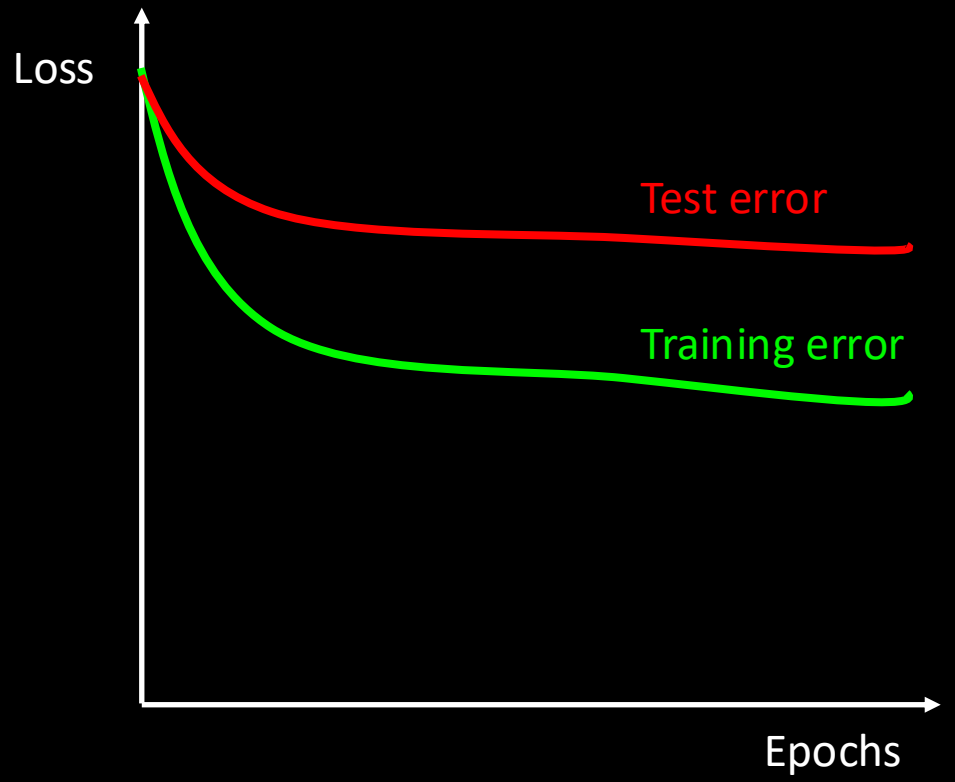
Model Capacity: Overfitting vs Underfitting



Can shift between underfitting and overfitting by adding/removing parameters

Ian Goodfellow, Yoshua Bengio, and Aaron Courville; Deep Learning, 2016

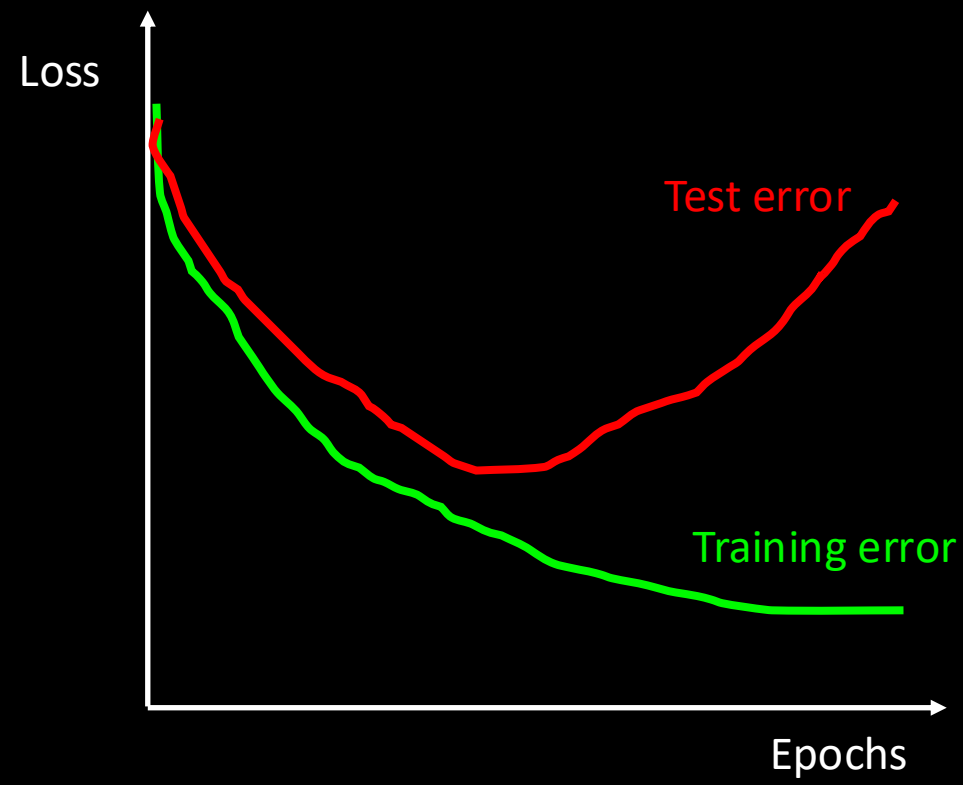
Model Capacity: Detecting Underfitting



Analyze **learning curves** for models tested on training data

- What happens to training data error as number of training steps increases?
  - Error remains high
- What happens to test data error as number of training steps increases?
  - Error remains high

Model Capacity: Detecting Overfitting



Analyze **learning curves** on training data and test data

- What happens to training data loss as number of training steps increases?
  - Error shrinks
- What happens to test data loss as number of training steps increases?
  - Error shrinks and then grows
- Why does training loss ***shrink*** and test loss ***grow***?
  - Models ***noise*** in the training data (i.e., "overfitting") at the expense of knowledge that generalizes
- What can cause noise in a dataset?
  - e.g., incorrect data entry/labeling, hardware measurement error (caution: some outliers are data points we want models to learn)

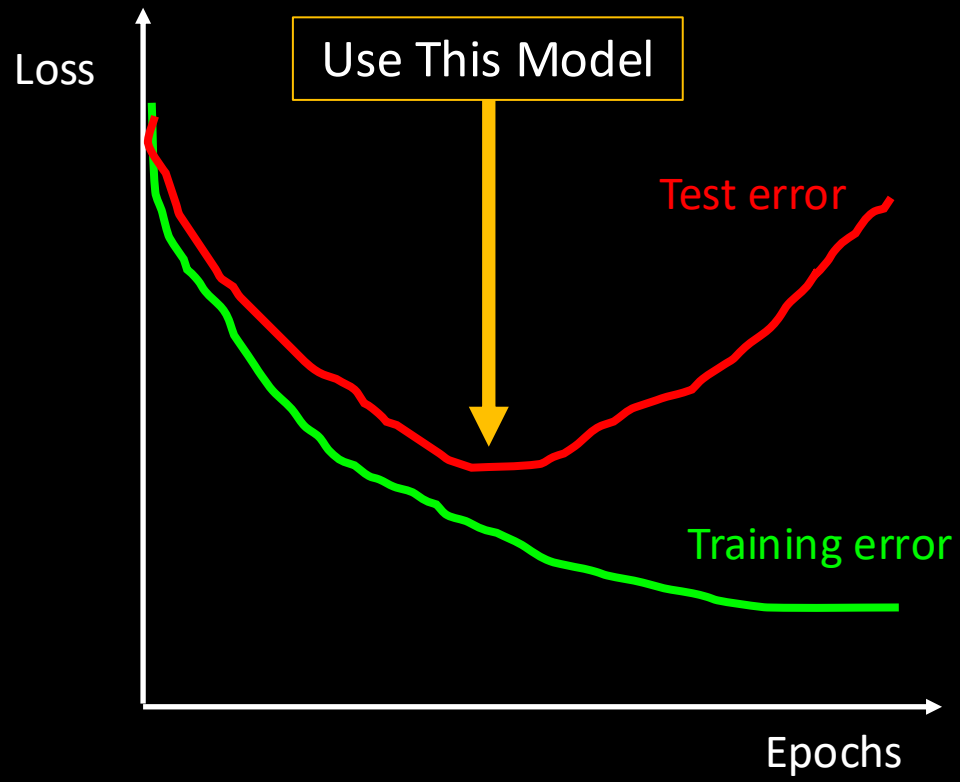
Often, we err on over-parameterized models capable of overfitting and then regularize them

Regularization

"any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error."

- Ch. 5.2 of Goodfellow Deep Learning book

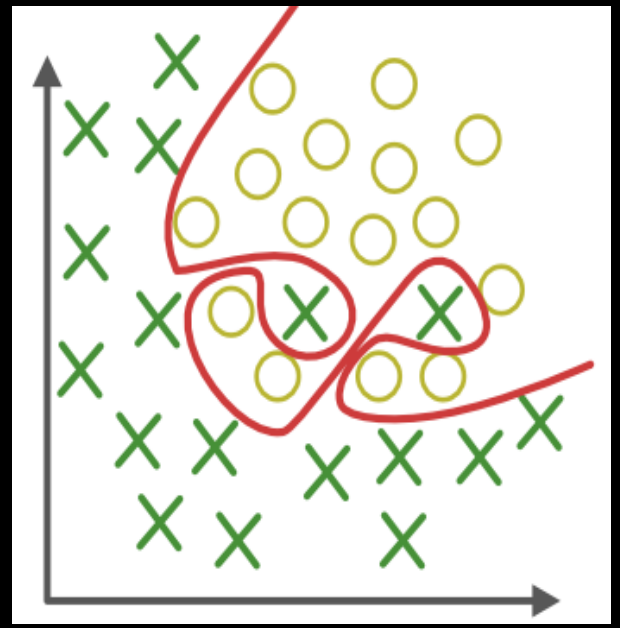
Regularizing Over-Parameterized Model: Early Stopping



Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)

Smooth a model's decision boundaries by incentivizing against large weights

(models can learn to represent noise with correlated large positive and negative weights that usually just cancel each other out)



https://towardsdatascience.com/underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6fe4a8a49dbf

Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)

Add penalty term to objective function; e.g., when *minimizing* sum of squared errors

- L2 norm: penalize squared weight values

$$Error = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} w_j^2$$

- L1 norm: penalize absolute weight values

$$Error = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} |w_j|$$

Analogy: Belt for Big Pants



- *Note: penalizes only weight (not biases) and can apply per layer or globally*

Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)

Add penalty term to objective function; e.g., when *minimizing* sum of squared errors

- L2 norm: penalize squared weight values

$$Error = \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} w_j^2$$

- L1 norm: penalize absolute weight values

$$Error = \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} |w_j|$$

Analogy: Belt for Big Pants



- Alpha determines contribution of norm penalty term (e.g., belt tightness)

Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)

Add penalty term to objective function; e.g., when *minimizing* **s**um of **s**quared **e**rrors

- L2 norm: penalize squared weight values

$$Error = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} w_j^2$$

- L1 norm: penalize absolute weight values

$$Error = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} |w_j|$$

Intuitively, larger alpha values prioritizes having weights closer to 0 instead of minimizing sum of squared errors

- Alpha determines contribution of norm penalty term (e.g., belt tightness)

Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)

Add penalty term to objective function; e.g., when *minimizing* **s**um of **s**quared **e**rrors

- L2 norm: penalize squared weight values

$$Error = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} w_j^2$$

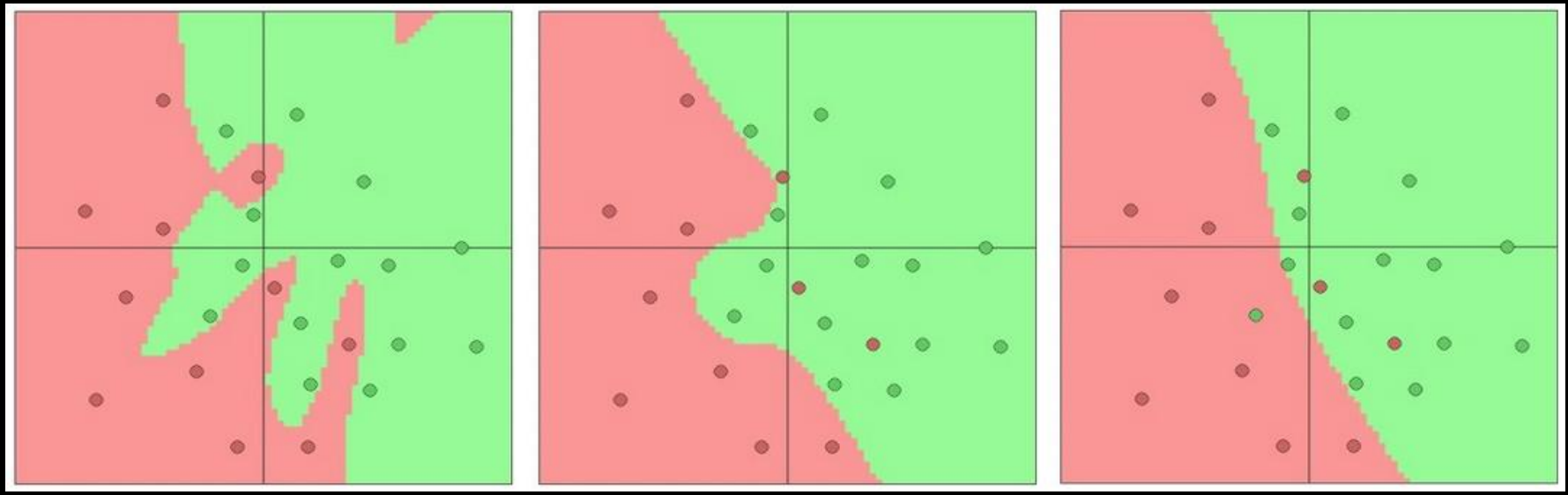
- L1 norm: penalize absolute weight values

$$Error = \sum_{i=1}^{n}(y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} |w_j|$$

- Learning impact: need gradient for weight penalty

Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)

(a) (b) (c)

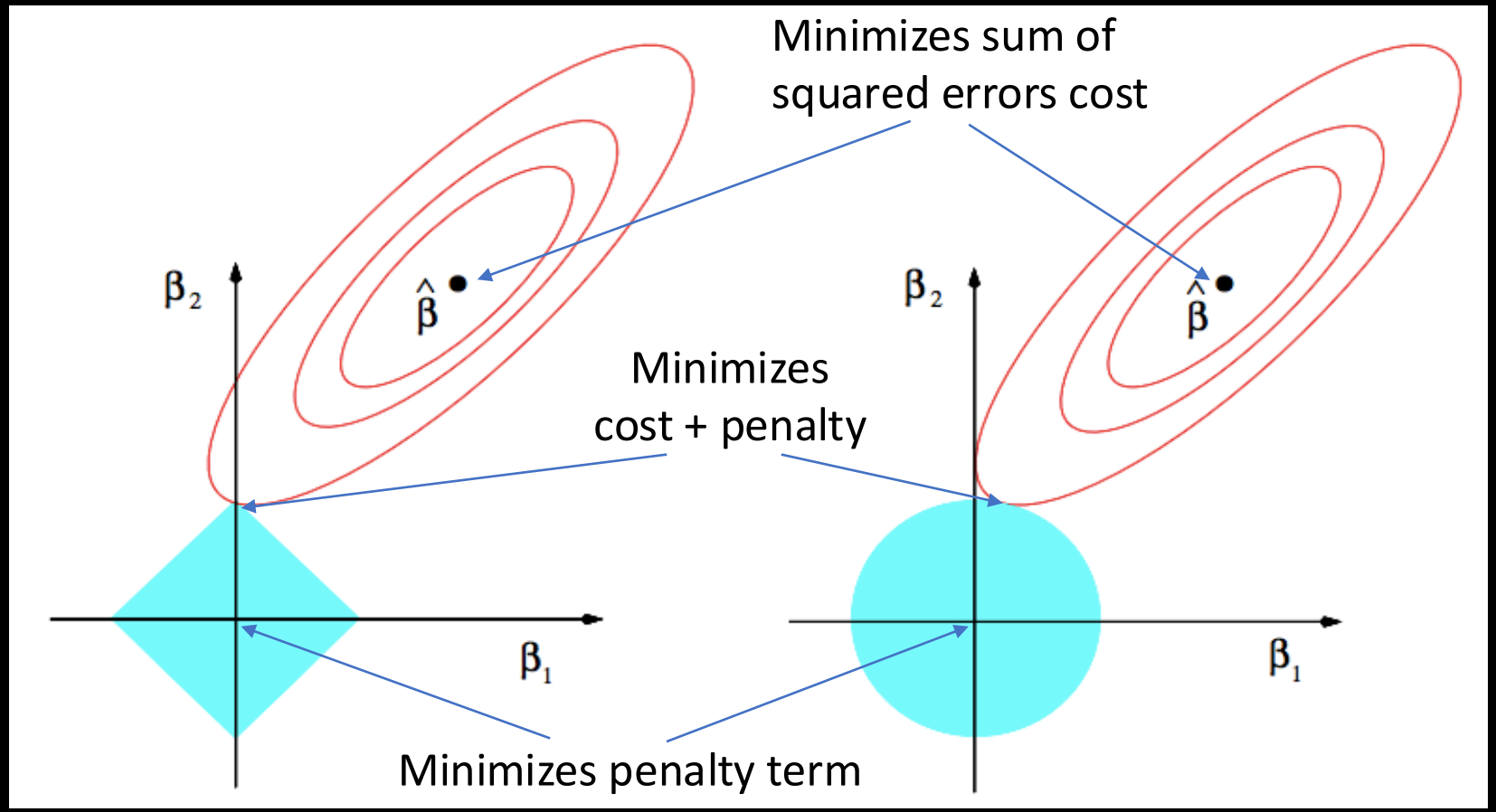


Which model had the largest value for alpha (i.e., norm penalty contribution), given that this value was the only difference when training the models?

$$Error = \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2 + \alpha \sum_{j=1}^{m} w_j^2$$

https://cs231n.github.io/neural-networks-1/

Regularizing Over-Parameterized Model: Weight Decay (Parameter Norm Penalty)



$|\beta_1| + |\beta_2| \leq t$

$\beta_1^2 + \beta_2^2 \leq t^2$

Note: L2 commonly used in practice

Contour of least square error function

Contour of regularization constraint functions shown below both plots

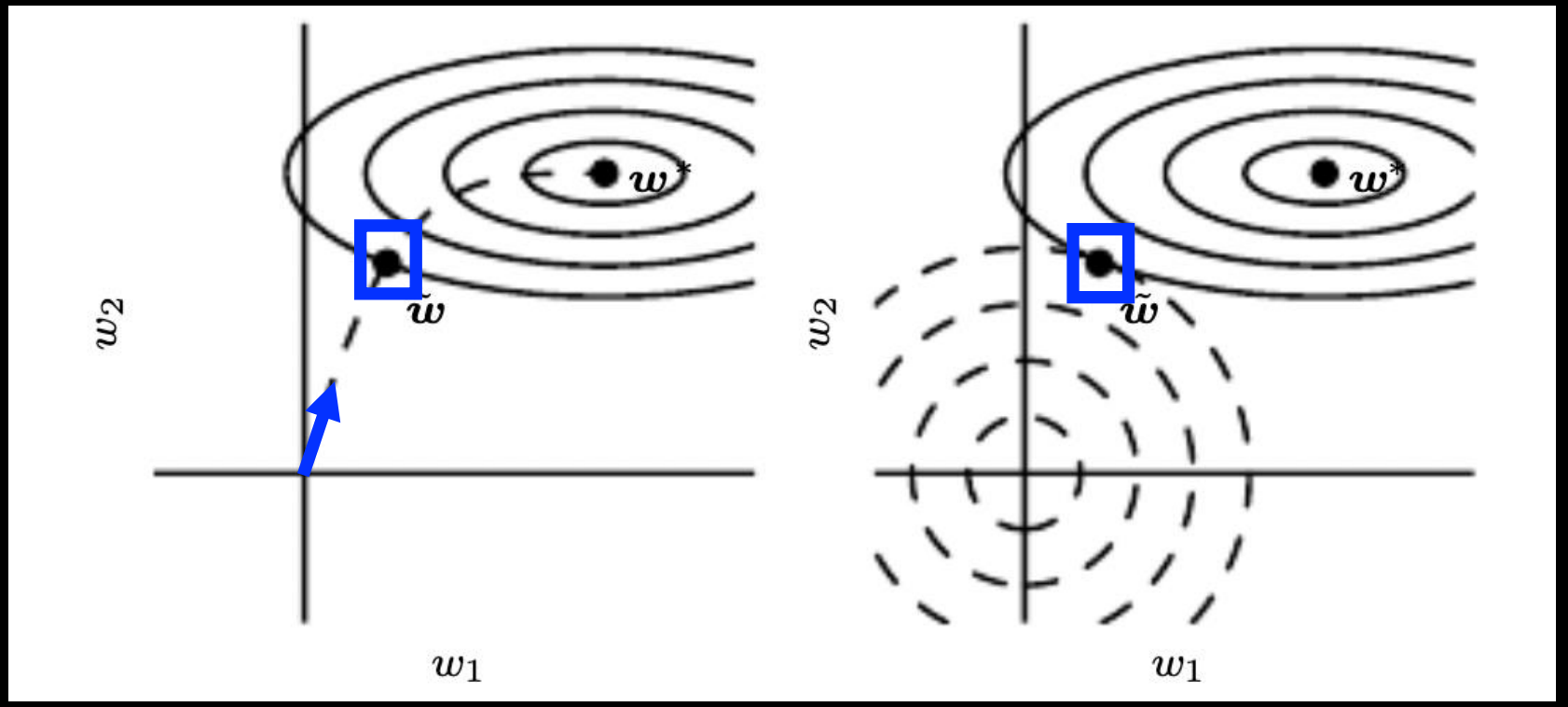
https://web.stanford.edu/~hastie/Papers/ESLII.pdf

Regularizing Over-Parameterized Model: Early Stopping vs Parameter Norm Penalties

Similar behavior when model parameters initialized around origin; e.g.,

Early Stopping

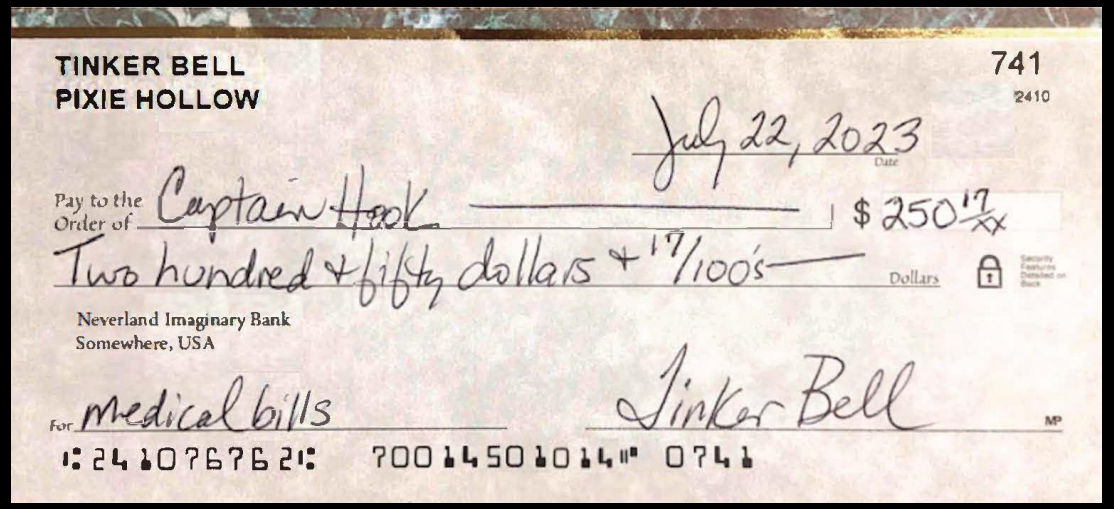
Parameter Norm Penalty (L2)



https://www.deeplearningbook.org/contents/regularization.html

Famous Success Stories for Shallow Learning

e.g., LeNet-5 reading millions of checks per month for banks



https://pirg.org/edfund/resources/how-to-fill-out-a-check-from-your-bank-account/

NETtalk popularized by the Today Show for its ability to speak written text



https://www.youtube.com/watch?v=gakJIr3GecE

Summary: while new learning techniques from the 2nd NN wave improved performance, neural networks still remained largely confined to research settings

Today's Topics

- Shallow learning: why doesn't it work?
- Navigating the loss landscape: how to make learning converge
- Generalizing to the real world: avoiding overfitting and underfitting
- Programming tutorial

Today's Topics

- Shallow learning: why doesn't it work?
- Navigating the loss landscape: how to make learning converge
- Generalizing to the real world: avoiding overfitting and underfitting
- Programming tutorial

The End